35. Docker Basics on Linux

35. Docker Basics on Linux

Phase: Phase 11 — Containers and cloud Linux
Estimated time: 2 days
Slug: `35-docker-basics`

Learning objectives

- Run containers interactively
- Understand images vs containers
- Mount volumes and publish ports
- Read container logs

Prerequisites

Complete prior numbered lessons. Prefer a real Linux VM/container for Linux-specific topics. Type every command yourself — muscle memory beats skimming.

Why this matters

Containers package an application with its userland dependencies so it runs the same on a laptop, a CI runner, and a server. Docker is the tool most people meet first. To use it safely you need to separate four ideas: **image**, **container**, **volume**, and **registry**, and to know the handful of `docker run` flags that matter, especially `--rm`, which keeps your host free of dead containers.

Core concepts

Term	Meaning
image	Read-only layered template (filesystem + metadata)
container	A running (or stopped) instance of an image
layer	One filesystem diff in an image; cached and shared
registry	Image store (Docker Hub, GHCR, private)
volume	Docker-managed persistent storage outside the container layer
bind mount	Host path mapped into the container
Dockerfile	Recipe to build an image
tag	Human label such as `nginx:1.27` (`latest` is just a tag)

```
Dockerfile --build--> image --run--> container
                        |              |
                     push/pull       writable layer (ephemeral)
                        v              |
                     registry      volume / bind mount (persistent)
```

Key `docker run` flags:

Flag	Meaning
`-d`	Detached (background)
`-it`	Interactive terminal
`--rm`	Remove container on exit
`-p H:C`	Publish host port H to container port C
`-v SRC:DST`	Mount volume or host path
`-e K=V`	Environment variable
`--name`	Fixed container name
`--restart`	Restart policy

`--rm` means **remove-on-exit**: when the container's main process stops, Docker deletes the container and its writable layer. Great for throwaway tests; wrong for something whose logs you need to inspect after a crash.

Worked examples

First containers:

```
docker run --rm hello-world
docker run --rm -it alpine:3.20 sh
docker run --rm ubuntu:24.04 cat /etc/os-release
```

Long-running service with a published port and a named volume:

```
docker run -d --name web -p 8080:80 -v webdata:/usr/share/nginx/html nginx:1.27
curl -sI http://127.0.0.1:8080 | head -n 1
docker ps
docker logs -f web
docker exec -it web sh
```

Inspect and clean:

```
docker inspect web --format '{{.State.Status}} {{.NetworkSettings.IPAddress}}'
docker stop web && docker rm web
docker images
docker image prune
docker system df
```

Build your own image:

```
FROM python:3.12-slim
WORKDIR /app
COPY app.py .
USER 10001
CMD ["python", "app.py"]
```

```
docker build -t myapp:0.1 .
docker run --rm myapp:0.1
```

Show the difference `--rm` makes:

```
docker run --name keep alpine true
docker run --rm --name gone alpine true
docker ps -a --filter name=keep --filter name=gone
# keep remains as Exited (0); gone does not exist
docker rm keep
```

Persistence check:

```
docker run --rm -v demo:/data alpine sh -c 'echo hi > /data/f'
docker run --rm -v demo:/data alpine cat /data/f      # hi
docker volume rm demo
```

Resource limits:

```
docker run --rm --memory 256m --cpus 0.5 alpine sh -c 'echo limited'
docker stats --no-stream
```

Mini-lab

1. Run `docker run --rm alpine echo hello` and then `docker ps -a`. Is the container still listed? Explain via `--rm`.
2. Run nginx on port 8080, curl it, read logs, exec into it, stop and remove.
3. Write a Dockerfile that prints a message. Build, run, and inspect layers with `docker history`.
4. Prove a container filesystem is ephemeral: write a file, remove the container (no volume), start another, look for the file.
5. Do the same with a named volume and show the data survives.

Decision guide / failure modes

Need	Use
One-off command or test	`docker run --rm`
Service	`-d --name --restart unless-stopped` + volume
Persist data	named volume or bind mount
Dev code reload	bind mount of source dir
Reproducible deploys	pin tags or digests, not `latest`

Failure modes:

- **Data loss**: databases in the container layer disappear with `docker rm`.
- **Root in container**: add `USER` in the Dockerfile; avoid `--privileged`.
- **Disk full**: unused images and build cache; `docker system df`, prune.
- **Port clash**: host port already in use; check `ss -ltnp`.
- **`latest` drift**: a rebuild silently changes the base image.
- **Debugging a crash with `--rm`**: the container is gone, so its logs are gone; drop `--rm` while investigating.
- **Secrets baked into images**: they stay in layers; pass at runtime.

Common mistakes

- docker.sock mounted into untrusted containers
- Running as root inside without need
- Storing secrets in images

Exercises

Work in `exercises/35-docker-basics/`. Read that folder's `README.md` fully before coding.

```
cd exercises/35-docker-basics
chmod +x test.sh exercise.sh
./test.sh
```

What you will build

Implement `docker_rm_flag_meaning` in `exercise.sh` so it prints a short hyphenated phrase describing what `docker run --rm` does when the container's main process ends. Use the exact phrase from the lesson, all lowercase with hyphens. Once the test passes, run the `keep` versus `gone` experiment from the Worked examples on a machine with Docker and write down which container remained and why. Keep the function body to one line; the goal is recall of the lesson vocabulary, not cleverness. Finish by re-reading the Decision guide table and noting one failure mode you could have caused yourself.

Study loop

- **A (Drill):** make `./test.sh` pass
- **B (Apply):** run 2–3 lab commands from this lesson on a real Linux host; paste notable output into your notes
- **C (Challenge):** rewrite the exercise from memory the next day

Do not open `solutions/35-docker-basics/` until you have an honest failing attempt.

Self-check

Try to answer without scrolling up:

1. Image vs container?
2. What does –rm do?

3. Why shared kernel matters?

Further reading

- https://docs.docker.com/get-started/